cQuizy

Fejlesztési Dokumentáció

Vizsgaremeket készítették:

Kele Márton Róbert

Molnár Károly

Tartalom

Tartalom....2
Specifikáció....3
1. Bevezetés....3
2. Rendszerkörnyezet....4
2.1. Futtatási környezet....4
2.2. Programozási környezet....4
3. Részletes specifikáció....4
3.1. Felhasználói szerepkörök....4
Csoportadminisztrátor....4
Csoporttag....4
3.2. Funkcionális követelmények....5
Felhasználókezelés....5
Csoportkezelés....5
Projekt- és tesztkészítés....5
Kérdéstípusok....5
Tesztírás folyamat....5
Értékelés és visszajelzés....6
3.3. Nem-funkcionális követelmények....6
Platform kompatibilitás....6
Biztonság....6
Teljesítmény....6
4. Adatlap....7
4.1 Projekt neve....7
4.2 Rövid ismertető....7
4.3 Fejlesztők....7
Adatbázis....8
1. ER Diagram....8
2. Adatbázi felépítése....9
2.1 Felhasználó adatai (users) [customuser]....9
2.2 Csoportok adatai (groups)....9
2.3 Csoport tagjainak adatai (groupMembers)....9
2.4 Jegyek százalékának adatai (gradePercentages)....10
2.5 Jegyek adatai (grades)....10
2.6 Projektek adatai (projects)....10
2.7 Tömbök adatai (blocks)....11
2.8 Opciók adatai (answers)....11
2.9 Kvízek adatai (quizzes)....11
2.10 Események adatai (events)....12
2.11 Beküldések adatai (submissions)....12
2.12 Beküldött válaszok adatai (submittedAnswers)....12

Tesztesetek....13
1. Frontend Tesztek....13
1.1. Regisztráció és bejelentkezés....13
2.2. Csoport és projekt létrehozás....14
2. Backend Tesztek....17
2.1. Felhasználók....17
2.2. Csoportok....21
2.3. Projektek....24
2.4. Kvízek....28

Specifikáció

1. Bevezetés

A cQuizy olyan alkalmazás, amivel a tanárok munkáját szeretnénk segíteni a diákok számonkérésében.

Az alkalmazás az oktatók számára olyan funkciókat biztosít, mint az egyszerű grafikus felületen történő dolgozatkészítés és az automatikus dolgozatjavítás, így a tanárnak nem kell a javítással foglalkoznia, mi ezt is megoldjuk helyette.

A szoftver elsődleges célja a digitális számonkérések integritásának garantálása. Ennek érdekében az alkalmazás figyeli a vizsgázó aktivitását: amennyiben a diák elhagyja az alkalmazás felületét vagy másik applikációra vált, a rendszer automatikusan zárolja a vizsgát. A dolgozat folytatása kizárólag a felügyelő tanár jóváhagyásával lehetséges.

Intézményi tulajdonú eszközökön (pl. tableteken) az alkalmazás futtatható dedikált vizsgamódban is. Ez a funkció korlátozza az eszköz használatát kizárólag a vizsgafelületre, ezzel teljes mértékben megakadályozva bármilyen külső szoftver vagy funkció elérését a számonkérés ideje alatt.

Ez az alkalmazás mind a tanárok munkáját, mind a diákokat segítené. Segítene a pedagógusnak dolgozatkészítésben és -javításban, valamint a csalásmentes, biztonságos számonkérésben, de a diákoknak is kis segítséget adhatna például a feleletválasztós tesztkérdés.

2. Rendszerkörnyezet

2.1. Futtatási környezet

- **Web**: bármely modern böngésző (Chrome, Firefox, Edge, stb.)
- **Windows** 10 vagy újabb, natív telepíthető kliens
- **Android** 9.0 vagy újabb, mobilalkalmazás

2.2. Programozási környezet

- **Frontend**: Dart + Flutter Framework
- **Backend**: Python + Django Framework + Django Ninja

3. Részletes specifikáció

3.1. Felhasználói szerepkörök

A cQuizy egyetlen, egységes fióktípust használ, így a regisztráció során a felhasználónak csupán egy felhasználónevet, e-mail címet és jelszót kell megadnia. A fiókoknak nincsenek előre meghatározott, fix jogosultságaik. A felhasználó jogköre dinamikusan változik: mindig attól függ, hogy az általa éppen használt csoportban adminisztrátori vagy egyszerű tagi szerepkörrel rendelkezik.

Csoportadminisztrátor

- Tagok felvétele és eltávolítása a csoportból
- Csoportra való szabályok beállítása (pl.: anticheat, kiosk, százalékok, stb.)
- Projektek létrehozása grafikus felületen és azok időzítése
- Nyomon követheti a tagok haladását a teszt során
- Tesztírás során értesítést kap, ha egy tag elhagyja az alkalmazást
    - Engedélyezheti vagy felfüggesztheti a tesztet
- Megtekintheti az automatikusan kijavított teszteket és átírhatja azokat
- Manuálisan felülbírálhatja a jegyeket
- Láthatja a csoport statisztikáját

Csoporttag

- Láthatja a számára aktív és jövőbeli teszteket
- Kitöltheti a tesztet annak aktív idejében
- A javítást követően megtekintheti eredményét
- Láthatja a saját statisztikai adatait és összehasonlíthatja az osztály összátlagával is

3.2. Funkcionális követelmények

Felhasználókezelés

- Regisztráció (felhasználónév, jelszó, e-mail cím)
- Bejelentkezés (felhasználónév vagy e-mail, jelszó)
- Profiladatok szerkesztése

Csoportkezelés

- Csoport létrehozása egyedi névvel
- A rendszer generál egyedi csatlakozási kódot
    - Mások ezzel csatlakozhatnak
    - A kód bármikor újragenerálható
- A csoportadmin kezelheti a tagokat és a csoport beállításait

Projekt- és tesztkészítés

- A csoportadmin hozhat létre projekteket egy csoporton belül. A projektek maguk a tesztek.
- Tesztkészítéshez egy egyszerű, de rugalmas szerkesztői felületet kínálunk.

Kérdéstípusok

- Feleletválasztós: Egy vagy több helyes válasz megjelölésének lehetősége.
- Szöveges bevitel: Rövid vagy hosszú szöveges válasz megadása. (Itt a javítás lehet automatikus, ha a válasz pontos egyezést igényel, vagy tanári értékelést igényelhet. Ezt a tanár tudja állítani, hogy pontos egyezésre automatikusan javítson, vagy addig ne zárhassa le a tanár a javítást, ameddig jóvá nem hagyta a beírt szöveget.)
- Lehetőség képek beillesztésére a kérdésekhez (linkről)
- A kérdések sorrendjének egyszerű módosítása
    - Véletlenszerű kérdéssorrend lehetősége

Tesztírás folyamat

- A diák a kezdési időpont után indíthatja el a tesztet.
- A rendszer egyértelműen mutatja a hátralévő időt.
- Csalás elleni védelem (anticheat): Ha a diák elhagyja az alkalmazást, az alkalmazás azonnal elküldi a beküldött válaszokat a szervernek és zárolja a tesztet. Ezt a tanár feloldhatja, de értékelheti az addig elért eredményt is.
- Kioszk Mód (kiosk): Ha ez a mód aktív, az alkalmazás teljes képernyős módban fut, és nem lehet elhagyni a teszt befejezéséig vagy az idő lejártáig.

Értékelés és visszajelzés

- A rendszer a teszt lezárása után a helyes válaszok alapján automatikusan kiszámolja a diák százalékos eredményét.
- A csoportnál beállított százalékos határok alapján a rendszer automatikusan érdemjegyet generál.
- A tanár egy felületen láthatja az összes diák eredményét, válaszait.
    - Láthat a csoportról statisztikákat.

3.3. Nem-funkcionális követelmények

Platform kompatibilitás

- A Flutter-kódbázisnak reszponzívan kell működnie webes, asztali (Windows) és mobil (Android) környezetben is.
- A felhasználói felületnek alkalmazkodnia kell a különböző képernyőméretehez.
- Az anti-cheat rendszer is platformonként változó módon, de feladatát ellátva képes működni.

Biztonság

- A jelszavakat és személyes adatokat hashelt formában tároljuk.
- Minden kommunikáció a kliens (Flutter) és a szerver (Django) között titkosított csatornán (HTTPS) történik.
- A szerveroldali jogosultságkezelés megakadályozza, hogy egy diák hozzáférjen más diák adataihoz vagy tanári funkciókhoz.

Teljesítmény

- Az alkalmazás gyors betöltődése
- Gyors kommunikáció a szerverrelFelhasználói felület (UI) és élmény (UX)
- Egyszerű, letisztult kezelőfelület mind a tanárok, mind a diákok számára.
- A fontos funkciók könnyen elérhetőek.
- Vizuális visszajelzések a felhasználói interakciókról
    - Vizuálisan megjelenített értesítés

4. Adatlap

4.1 Projekt neve

cQuizy

4.2 Rövid ismertető

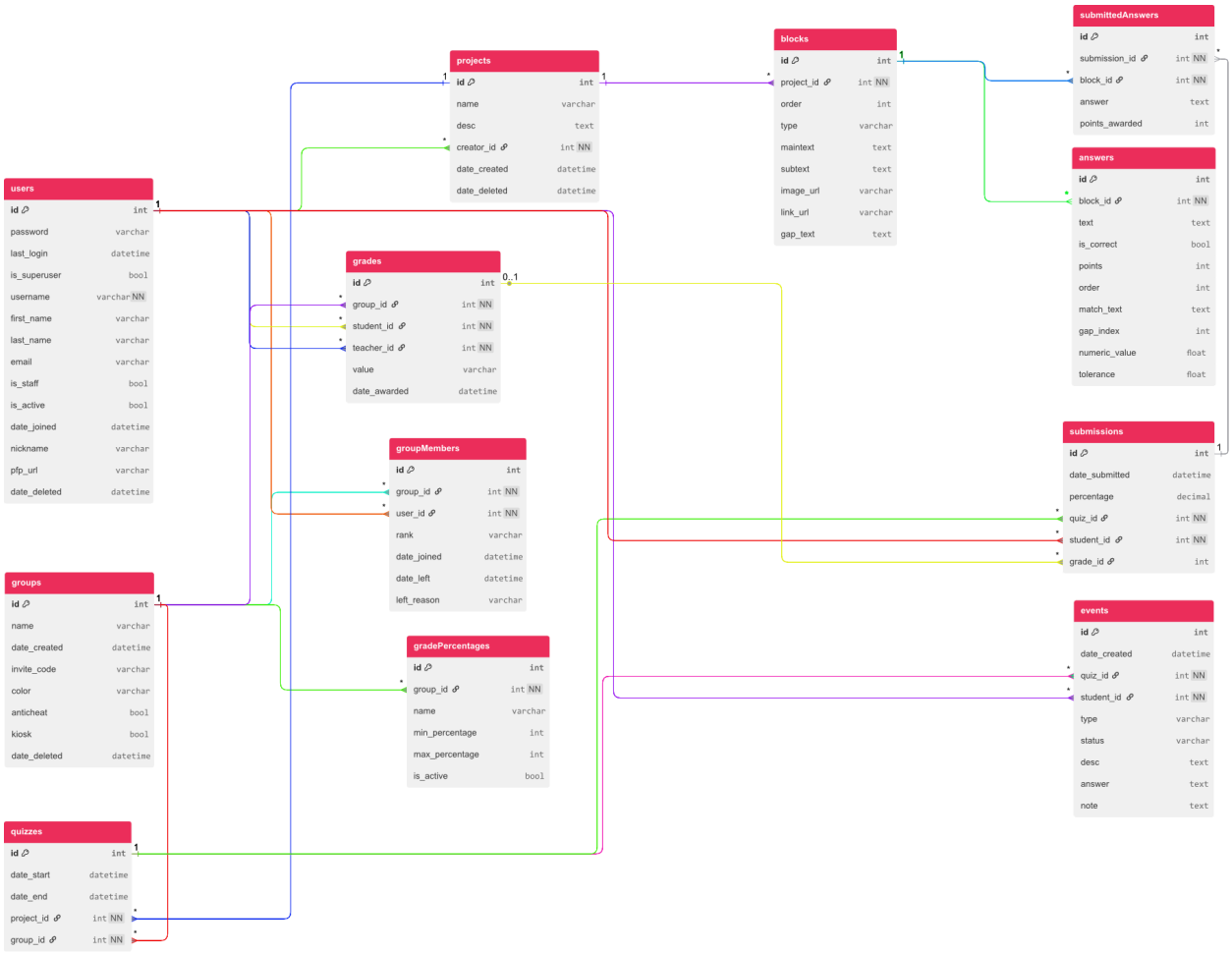
A cQuizy egy oktatási célú alkalmazás, amely lehetővé teszi a tanárok számára dolgozatok egyszerű elkészítését és automatikus javítását. A beépített csalás elleni funkciók garantálják a számonkérések tisztaságát, a többplatformos elérhetőség pedig rugalmas használatot biztosít mind tanároknak, mind diákoknak.

4.3 Fejlesztők

- Kele Márton Róbert Backend fejlesztő
- Molnár Károly Frontend fejlesztő

Adatbázis

1. ER Diagram



2. Adatbázi felépítése

2.1 Felhasználó adatai (users) [customuser]

- `id` | Azonosító, ellenőrzésre szolgál
- `password` | Jelszó, titkosított (Hash)
- `last_login` | Mikor jelentkezett be utoljára a felhasználó?
- `is_superuser` | A felhasználó admin e?
- `username` | Felhasználónév, bármi lehet, két ugyanolyan nem létezhet
- `first_name` | Felhasználó keresztneve
- `last_name` | Felhasználó családneve
- `email` | E-mail
- `is_staff` | A felhasználó moderátor e?
- `is_active` | Aktív-e a felhasználó profilja?
- `date_joined` | Mikor lett létrehozva a profil?
- `nickname` | Becenév, nem kötelező megadni
- `pfp_url` | A profilképeket egy third party oldalon tároljuk és az onnan származó linket tároljuk.
- `date_deleted` | Megjelöli a törölt csoportokat egy dátummal

2.2 Csoportok adatai (groups)

- `id` | Azonosító
- `date_created` | Csoport létrehozásának dátuma
- `name` | Csoport neve
- `invite_code` | Meghívó kód
- `color` | Csoport színe
- `anticheat` | Bool érték, he engedélyezve van, a kvíz letiltja a tanulót ha elhagyja az alkalmazást.
- `kiosk` | Bool érték, ha engedélyezve van a diák nem tudja elhagyni a kvízt.
- `date_deleted` | Megjelöli a törölt csoportokat egy dátummal

2.3 Csoport tagjainak adatai (groupMembers)

- `id` | Azonosító
- `group_id` | Csoport azonosítója
- `user_id` | Felhasználó azonosítója
- `rank` | Felhasználó rankja (Admin/Member)
- `date_joined` | Mikor csatlakozott a csoporthoz?
- `date_left` | Megjelöli a kilépett tagokat egy dátummal
- `left_reason` | Miért lépett ki a felhasználó? (LEFT, KICKED, GROUP_DELETED)

2.4 Jegyek százalékának adatai (gradePercentages)

- `id` | Azonosító
- `group_id` | Csoport azonosítója
- `name` | Jegy elnevezése
- `min_percantage` | Jegy megadás százalékának alsó határa
- `max_percantage` | Jegy megadás százalékának felső határa
- `is_active` | Törölt jegyek már nem aktívak, de múltbeli kvízeknél ugyanúgy megjelennek.

2.5 Jegyek adatai (grades)

- `id` | Azonosító
- `date_awarded` | Jegy létrehozásának dátuma
- `value` | Jegy értéke
- `student_id` | A diák azonosítója akié a jegy
- `teacher_id` | A tanár azonosítója aki adta a jegyet
- `group_id` | A csoport azonosítója, ahová került a jegy

2.6 Projektek adatai (projects)

- `id` | Azonosító
- `date_created` | Projekt létrehozásának dátuma
- `name` | Projekt neve
- `desc` | Project leírása
- `creator_id` | A létrehozó azonosítója
- `date_deleted` | Megjelöli a törölt projekteket egy dátummal

❓ **Megjegyzés:** Alapból a projektek nem tartoznak csoportokhoz, a létrehozó profiljához fognak tartozni. Ezeket később beszúrhatja kvíz ként olyan csoportokba, ahol adminisztrációs jogai vannak.

❗ **Fejlesztési lehetőség a jövőben:** Egy Project Hub ahol mindenki megoszthatja a projekteket amiket készítettek és ezt mások beszúrhatják a csoportjukba. Ehhez érdemes lesz a táblát bővíteni adatokkal mint például "category", "tags", stb.

2.7 Tömbök adatai (blocks)

- `id` | Azonosító
- `project_id` | Melyik projektnek része a tömb? (Projekt azonosítója)
- `order` | Hányadik kérdés a kvízen belül?
- `type` | Tömb fajtája (pl.: Szöveges, Feleletválasztós, stb)
- `question` | Tömb kérdése
- `subtext` | Opcionális feladatleírás vagy egyéb információ
- `gap_text` | Szövegkiegészítős tömbök alapja.
- `image_url` | Opcionális képfeltöltés
- `link_url` | Opcionális linkbeszúrás

2.8 Opciók adatai (answers)

- `id` | Azonosító
- `block_id` | Melyik tömbhöz tartozik?
- `order` | Az opció helyzetét határozza meg.
- `text` | Az opció szövege (Üres ha a tömb fajtája szöveges választ igényel)
- `match_text` | Párosítós feladatoknál, maga a "pár".
- `gap_index` | Megadja, hogy a válasz melyik szövegközi résbe illik.
- `numeric_value` | Becslés alapú feladatokhoz (szám).
- `tolarance` | Becslés alapú feladatokhoz (tolerancia).
- `is_correct` | Bool érték, hogy a válasz helyes vagy nem? (Szöveges kérdés esetén jelöli a helyes választ.)
- `points` | Hány pontot ér a megjelölt válasz?
- `grade_id` | Közvetlen kapcsolat jött létre a beküldés és a kapott érdemjegy között (opcionális).

2.9 Kvízek adatai (quizzes)

- `id` | Azonosító
- `date_start` | Pontos kezdete a kvíznek
- `date_end` | Pontos vége a kvíznek
- `project_id` | Azonosítója a megoldandó projektnek
- `group_id` | Azonosítója a csoportnak, akik megoldják

2.10 Események adatai (events)

- `id` | Azonosító
- `date_created` | Mikor történt az esemény?
- `quiz_id` | Melyik kvíznél történt?
- `student_id` | Melyik tanulóhoz tartozik?
- `type` | Mit takar az esemény? (gépi olvasáshoz)
- `status` | Egy esemény lehet "static", "active", vagy "handled"
    - *static* : Csak visszanézésre szolgál
    - *active* : Például mikor egy diákot kizár a rendszer a tanár ezt látja és feloldhatja
    - *handled* : Amikor egy "active" eseményt a tanár lekezelt
- `desc` | Leírása az eseménynek
- `answer` | Nem kötelező mező, a tanár válaszát továbbítja ha van (gépi olvasáshoz)
- `note` | Ha a tanár szeretne társíthat hozzá üzenetet

2.11 Beküldések adatai (submissions)

- `id` | Azonosító
- `date_submitted` | Mikor lett a kvíz leadva?
- `percentage` | Hány százalékot ért el a diák?
- `quiz_id` | A kvíz azonosítója
- `student_id` | Ki írta a kvízt?
- `grade_id` | Melyik jegy tartozik ehhez a beküldéshez?

2.12 Beküldött válaszok adatai (submittedAnswers)

- `id` | Azonosító
- `submission_id` | Melyik beküldés része a válasz? (Annak azonosítója)
- `block_id` | Melyik tömbre a válasz a kérdés? (Annak azonosítója)
- `answer` | A megjelölt válasz szöveg ként
- `points_awarded` | A kérdésre adott pontszám (automatikus, de a tanár átírhatja)

Tesztesetek

1. Frontend Tesztek

A Flutter teszteléséhez a beépített tesztelőt használtam amihez a teszteket szintén dart programozási nyelvben kell írni.

1.1. Regisztráció és bejelentkezés

Leírás	Autentikációs folyamatot (Regisztráció, Bejelentkezés) az API-n keresztül.
Előfeltétel	API szerver fut és elérhető
Várt eredmény	Sikeres bejelentkezés és token lekérése
Eredmény	Sikeres

Register and Login flow with test_0 Authentication Flow Integration Test < test\auth_test.dart < cquizy

Golden Test

Leírás	Login képernyő vizuális megjelenését
Előfeltétel	nincs előfeltétel
Várt eredmény	A képernyő megjelenése megegyezik a referenciaképpel.
Eredmény	Sikeres

LoginPage Golden Test - Initial State test\golden_login_test.dart < cquizy

2.2. Csoport és projekt létrehozás

Leírás	1. Bejelentkezik egy létező felhasználóval a rendszerbe, amivel szerez egy hozzáférési tokent. 2. Létrehoz egy új Projektet az API-n keresztül, és ellenőrzi, hogy sikeres volt-e.
Előfeltétel	AZ API szerver fut és elérhető a konfigurált címen, illetve létezik egy "test_0" nevű tesztfelhasználó.
Várt eredmény	Minden végpont visszajelzése 200-as vagy 201-es HTTP kód (vagy logikai siker), és a létrehozott entitások adatai megegyeznek a beküldöttel.
Eredmény	Sikeres

1. Bejelentkezés a szerverre (test_0 userrel) Project és Csoport Létrehozás Integrációs Teszt ‹ test\integration_test.dart ‹ cquizy
2. Új Projekt létrehozása API-n keresztül Project és Csoport Létrehozás Integrációs Teszt ‹ test\integration_test.dart ‹ cquizy

Leírás	3. Létrehoz egy új Csoportot az API-n keresztül, és validálja annak sikerességét (pl. pontos színkód ellenőrzése).
Előfeltétel	AZ API szerver fut és elérhető a konfigurált címen, illetve létezik egy "test_0" nevű tesztfelhasználó.
Várt eredmény	Minden végpont visszajelzése 200-as vagy 201-es HTTP kód (vagy logikai siker), és a létrehozott entitások adatai megegyeznek a beküldöttel.
Eredmény	Sikeres

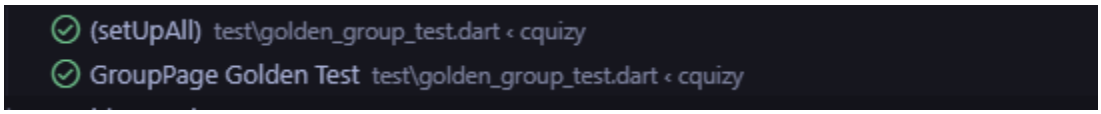
3. Új Csoport létrehozása API-n keresztül Project és Csoport Létrehozás Integrációs Teszt ‹ test\integration_test.dart ‹ cquizy

Felületi vizuális tesztek:

Leírás	A ProjectEditorPage megjelenését teszteli
Előfeltétel	nincs előfeltétel
Várt eredmény	A képernyő megjelenése megegyezik a referenciaképpel.
Eredmény	Sikeres

Dart
(setUpAll) test\golden_project_test.dart ‹ cquizy
ProjectEditorPage Golden Test test\golden_project_test.dart ‹ cquizy

Leírás	A GroupPage vizuális megjelenését teszteli
Előfeltétel	nincs előfeltétel
Várt eredmény	A képernyő megjelenése megegyezik a referenciaképpel.
Eredmény	Sikeres



2. Backend Tesztek

A backend tesztek pytest segítségével lettek elkészítve.

2.1. Felhasználók

Felhasználó regisztrálása

Végpont	`POST /api/users/register`
Leírás	Új felhasználó regisztrálása érvényes adatokkal.
Előfeltétel	A megadott felhasználónév és email nem szerepel az adatbázisban.
Teszt	

```python
def test_register_user_success(client):
    """Test successful user registration."""
    payload = {
        "username": "new_user",
        "email": "new@example.com",
        "password": "securepassword",
        "first_name": "john",
        "last_name": "doe",
        "nickname": "Johnny"
    }

    response = client.post("/register", json=payload)

    assert response.status_code == 200
    assert "success" in response.json()

    #? Felhasználó létrejött-e
    user = User.objects.get(username="new_user")
    assert user.email == "new@example.com"

    #? Név helyesen lett-e formázva
    assert user.first_name == "John"
```

Várt Eredmény	HTTP 200-as státuszkód. Az új felhasználó bekerül az adatbázisba és a neve automatikusan megformázódik.
Eredmény	Sikeres

Bejelentkezés

Végpont	`POST /api/users/login`
Leírás	Létező felhasználó bejelentkeztetése és hitelesítése JWT-tokennel.
Előfeltétel	A felhasználó létezik, érvényes adatokkal lép be.
Teszt	

```
def test_login_success(client, test_user):
    """Test login with correct credentials returns a
token."""
    payload = {
        "username": "testuser_123",
        "password": "strongpassword123"
    }

    response = client.post("/login", json=payload)

    assert response.status_code == 200

    #? Token létrejött-e
    assert "token" in response.json()
```

Várt Eredmény	HTTP 200-as státuszkód. A JWT-tokent visszakapja a felhasználó.
Eredmény	Sikeres

Felhasználó adatainak védelme

Végpont	`POST /api/users/me`
Leírás	Annak ellenőrzése, hogy az API blokkolja a token nélküli kéréseket a védett végpontokon.
Előfeltétel	A lekéréshez nincs csatolva a JWT-token.

Teszt	`def test_get_me_unauthenticated(client):`
	`    """Test getting current user data fails without a token."""`
	`    response = client.get("/me")`
	
	`    assert response.status_code == 401`

Várt Eredmény	HTTP 401-as státuszkód. A lekérést a backend elutasítja.
Eredmény	Sikeres

Végpont	`POST /api/users/me`
Leírás	Annak ellenőrzése, hogy a bejelentkezett felhasználó saját adatait sikeresen le tudja kérni.
Előfeltétel	A lekéréshez csatolva lett a JWT-token.

Teszt	`def test_get_me_authenticated(client, auth_headers, test_user):`
	`    """Test getting current user data with a valid JWT token."""`
	`    response = client.get("/me", headers=auth_headers)`
	
	`    assert response.status_code == 200`
	`    data = response.json()`
	`    assert data["username"] == test_user.username`
	`    assert data["email"] == test_user.email`

Várt Eredmény	HTTP 200-as státuszkód. A lekérést a backend így már elfogadja.
Eredmény	Sikeres

Felhasználó törlése

Végpont	`DELETE /api/users/me`
Leírás	A fióktörlés ellenőrzése a jelszó megadása után, illetve a "soft-delete" végrehajtásának ellenőrzése.
Előfeltétel	A felhasználó be van jelentkezve, van érvényes tokenje és a kérés törzsében a helyes jelszót küldi el.

Teszt	`def test_delete_account(client, auth_headers, test_user):`
	`    """Test soft-deletion of the user account."""`
	`    payload = {`
	`        "password": "strongpassword123"`
	`    }`
	
	`    response = client.delete("/me", json=payload, headers=auth_headers)`
	
	`    assert response.status_code == 200`
	
	`    # Felhasználó újratöltése, létezik e még`
	`    test_user.refresh_from_db()`
	
	`    assert test_user.is_active is False`
	`    assert test_user.date_deleted is not None`
	`    #? E-mail anonymizálva lett-e`
	`    assert test_user.email != "test@email.com"`

Várt Eredmény	Az adatbázisban a rekord nem törlődik fizikailag, hanem az `is_active` mező `False`-ra vált, a `date_deleted` kitöltésre kerül a jelenlegi időbélyeggel, az e-mail cím és a nevek pedig anonimizálásra kerülnek.
Eredmény	Sikeres

2.2. Csoportok

Csoport létrehozása

Végpont	`POST /api/groups`
Leírás	Új csoport létrehozása. A teszt ellenőrzi a végpont működését és a jogosultságok automatikus kiosztását.
Előfeltétel	A felhasználó be van jelentkezve (érvényes JWT token).

Teszt	`def test_create_group(groups_client, auth_headers_groups, test_user_groups):`
	`    """Test creating a new group."""`
	`    payload = {`
	`        "name": "Matek 6/B",`
	`        "color": "#FF5733"`
	`    }`
	
	`    response = groups_client.post("/", json=payload, headers=auth_headers_groups)`
	
	`    assert response.status_code == 200`
	`    data = response.json()`
	`    assert data["name"] == "Matek 6/B"`
	`    assert data["color"] == "#FF5733"`
	`    assert "invite_code" in data`
	
	`    group = Group.objects.get(id=data["id"])`
	`    member = GroupMember.objects.get(group=group, user=test_user_groups)`
	`    assert member.rank == "ADMIN"`

Várt Eredmény	HTTP 200-as státuszkód. A válasz tartalmazza a generált meghívó kódot, az adatbázisban pedig létrejön a kapcsolat, ahol a létrehozó ADMIN rangot kap.
Eredmény	Sikeres

Csoporthoz csatlakozás

Végpont	`POST /api/groups/join`
Leírás	Többfelhasználós folyamat tesztelése: "A" felhasználó létrehoz egy csoportot, "B" felhasználó pedig csatlakozik hozzá a generált kód alapján.
Előfeltétel	Létezik egy "Kémia 8/A" nevű csoport. A 2. felhasználó be van jelentkezve, ismeri a kódot, de még nem tagja a csoportnak.

Teszt	`def test_join_group(groups_client, auth_headers_groups, test_user_groups, auth_headers_2_groups, test_user_2_groups):`
	`    """Test joining a group using a valid invite code."""`
	`    payload = {"name": "Kémia 8/A", "color": "#0000FF"}`
	`    create_response = groups_client.post("/", json=payload, headers=auth_headers_groups)`
	`    invite_code = create_response.json()["invite_code"]`
	`    group_id = create_response.json()["id"]`
	
	`    join_payload = {"invite_code": invite_code}`
	`    join_response = groups_client.post("/join", json=join_payload, headers=auth_headers_2_groups)`
	
	`    assert join_response.status_code == 200`
	`    assert join_response.json()["name"] == "Kémia 8/A"`
	
	`    group = Group.objects.get(id=group_id)`
	`    member = GroupMember.objects.get(group=group, user=test_user_2_groups)`
	`    assert member.rank == "MEMBER"`

Várt Eredmény	HTTP 200-as státuszkód a csatlakozáskor. A 2. felhasználó bekerül a csoportba, és az adatbázisban automatikusan MEMBER (tag) rangot kap.
Eredmény	Sikeres

Csoport módosítása

Végpont	`PATCH /api/groups/{id}`
Leírás	Egy létező csoport adatainak (név, színkód) részleges frissítése.
Előfeltétel	A csoport létezik, és a kérést küldő felhasználó a csoport adminisztrátora.

Teszt	`def test_update_group_settings(groups_client, auth_headers_groups, test_user_groups):`
	`    """Test updating a group's settings (requires ADMIN rank)."""`
	`    payload = {"name": "Történelem 11/I", "color": "#333333"}`
	`    create_response = groups_client.post("/", json=payload, headers=auth_headers_groups)`
	`    group_id = create_response.json()["id"]`
	
	`    update_payload = {"name": "Történelem 12/I", "color": "#444444"}`
	`    update_response = groups_client.patch(f"/{group_id}", json=update_payload, headers=auth_headers_groups)`
	
	`    assert update_response.status_code == 200`
	`    data = update_response.json()`
	`    assert data["name"] == "Történelem 12/I"`
	`    assert data["color"] == "#444444"`

Várt Eredmény	HTTP 200-as státuszkód. A válaszban az új, frissített név ("Történelem 12/I") és az új színkód ("#444444") szerepel, a módosítások pedig elmentődnek.
Eredmény	Sikeres

2.3. Projektek

Projekt létrehozása

Végpont	`POST /api/projects`
Leírás	Új, egyelőre üres projekt létrehozása. A teszt ellenőrzi, hogy a rendszer automatikusan a kérést küldő felhasználóhoz rendeli-e (creator) a rekordot.
Előfeltétel	A felhasználó be van jelentkezve (érvényes JWT token).

Teszt	`def test_create_project(blueprints_client, auth_headers_blueprints, test_user_blueprints):`
	`    """Test successful project creation."""`
	`    payload = {`
	`        "name": "Test Project",`
	`        "desc": "This is a test project"`
	`    }`
	
	`    response = blueprints_client.post("/", json=payload, headers=auth_headers_blueprints)`
	
	`    assert response.status_code == 200`
	`    data = response.json()`
	`    assert data["name"] == "Teszt Projekt"`
	
	`    project = Project.objects.get(id=data["id"])`
	`    assert project.name == "Teszt Projekt"`
	`    assert project.creator == test_user_blueprints`

Várt Eredmény	HTTP 200-as státuszkód. Az adatbázisban létrejön a projekt a megadott névvel és leírással, az alkotó (creator) pedig az aktuális tesztfelhasználó lesz.
Eredmény	Sikeres

Projekt részleteinek lekérdezése

Végpont	`GET /api/projects/{id}`
Leírás	Egy adott projekt adatainak és a hozzá tartozó beágyazott kérdéseknek/válaszoknak a lekérése azonosító (ID) alapján.
Előfeltétel	Létezik az adatbázisban a projekt, és azt a lekérdezést indító bejelentkezett felhasználó hozta létre.

Teszt	`def test_get_project_details(blueprints_client, auth_headers_blueprints, test_user_blueprints):`
	`    """Test retrieving full project details."""`
	`    project = Project.objects.create(name="Projekt", creator=test_user_blueprints)`
	
	`    response = blueprints_client.get(f"/{project.id}/", headers=auth_headers_blueprints)`
	
	`    assert response.status_code == 200`
	`    data = response.json()`
	`    assert data["id"] == project.id`
	`    assert data["name"] == "Projekt"`

Várt Eredmény	HTTP 200-as státuszkód. A JSON válaszban a megfelelő projekt adatai (ID, név) térnek vissza.
Eredmény	Sikeres

Projekt frissítése

Végpont	`PUT /api/projects/{id}`
Leírás	Beágyazott (nested) adatszerkezetek frissítése: A teszt ellenőrzi, hogy egy kérésen belül képes-e a rendszer frissíteni a projekt nevét, létrehozni egy új kérdésblokkot, generálni hozzá válaszokat, valamint lekezeli-e a frontend ("single") és a backend ("single_choice") típusok közötti konverziót.
Előfeltétel	A projekt létezik. A payload egy összetett struktúrát tartalmaz (projekt név + 1 kérdés + 2 válasz).
Teszt	

```
def
test_update_full_project_with_blocks(blueprints_client,
auth_headers_blueprints, test_user_blueprints):
    """Test complex update logic: replacing project name
and managing blocks/answers."""
    project = Project.objects.create(name="Régi Név",
creator=test_user_blueprints)

    payload = {
        "name": "Új Név",
        "desc": "Új Leírás",
        "blocks": [
            {
                "type": "single",
                "maintext": "Mennyi 2+2?",
                "subtext": "",
                "answers": [
                    {"text": "3", "is_correct": False,
"points": 0},
                    {"text": "4", "is_correct": True,
"points": 1}
                ]
            }
        ]
    }

    response = blueprints_client.put(f"/{project.id}/",
json=payload, headers=auth_headers_blueprints)
```

	<pre>assert response.status_code == 200
	data = response.json()
	assert data["name"] == "Új Név"
	
	project.refresh_from_db()
	assert project.name == "Új Név"
	assert project.blocks.count() == 1
	
	block = project.blocks.first()
	assert block.type == "single_choice"
	assert block.maintext == "What is 2+2?"
	assert block.answers.count() == 2</pre>

Várt Eredmény	HTTP 200-as státuszkód. Az adatbázis ellenőrzésekor a projekt neve sikeresen megváltozott, létrejött egy `single_choice` típusú kérdésblokk, amihez pontosan 2 válasz rekord tartozik.
Eredmény	Sikeres

2.4. Kvízek

Új kvíz létrehozása

Végpont	`POST /api/quizzes/`
Leírás	Tanári/Admin funkció tesztelése: egy meglévő tesztsablon (Project) hozzárendelése egy csoporthoz (Group) egy konkrét időablakkal (kezdés és befejezés).
Előfeltétel	A kérést küldő felhasználó a csoport ADMIN-ja (tanár). A kezdési és befejezési dátumok logikailag helyesek (a befejezés a kezdés után van).

Teszt	`def test_create_quiz(quizzes_client, teacher_auth_headers, test_group, test_project):`
	`    """Test successful quiz creation by a teacher."""`
	`    start_time = timezone.now() + timedelta(days=1)`
	`    end_time = start_time + timedelta(hours=1)`
	
	`    payload = {`
	`        "group_id": test_group.id,`
	`        "project_id": test_project.id,`
	`        "date_start": start_time.isoformat(),`
	`        "date_end": end_time.isoformat()`
	`    }`
	
	`    response = quizzes_client.post("/", json=payload, headers=teacher_auth_headers)`
	
	`    assert response.status_code == 200`
	`    data = response.json()`
	`    assert data["project"]["name"] == "Teszt Projekt"`
	
	`    quiz = Quiz.objects.get(id=data["id"])`
	`    assert quiz.group == test_group`
	`    assert quiz.project == test_project`

Várt Eredmény	HTTP 200-as státuszkód. Az adatbázisban a Quiz táblában létrejön a rekord, ami sikeresen összeköti a projektet, a csoportot és az időpontokat.
Eredmény	Sikeres

Kvíz lekérése és indítása (diáknak)

Végpont	`GET /api/quizzes/{id}/start`
Leírás	Annak tesztelése, hogy a diák megkapja-e a vizsga kérdéseit, és hogy az adatok biztonságosan (a helyes megoldásokat elrejtve) érkeznek-e meg a frontend felé.
Előfeltétel	A vizsga státusza aktív (a jelenlegi időpont a start és end dátum közé esik), a kérést küldő diák pedig tagja a csoportnak, de még nem küldte be a vizsgát.

Teszt	`def test_start_quiz(quizzes_client, student_auth_headers, test_group, test_project):`
	`    """Test student getting quiz questions (start quiz)."""`
	`    now = timezone.now()`
	`    quiz = Quiz.objects.create(`
	`        group=test_group,`
	`        project=test_project,`
	`        date_start=now - timedelta(minutes=5),`
	`        date_end=now + timedelta(hours=1)`
	`    )`
	
	`    response = quizzes_client.get(f"/{quiz.id}/start", headers=student_auth_headers)`
	
	`    assert response.status_code == 200`
	`    data = response.json()`
	`    assert data["project"]["name"] == "Teszt Projekt"`
	`    assert len(data["project"]["blocks"]) == 1`
	
	`    assert data["project"]["blocks"][0]["maintext"] == "1+1?"`

Várt Eredmény	HTTP 200-as státuszkód. A válasz JSON-ben szerepelnek a kérdések és a válaszlehetőségek, de a szenzitív adatok (pl. `is_correct` mező) a sémának köszönhetően nem kerülnek kiküldésre.
Eredmény	Sikeres

Kvíz beadása és automatikus értékelése

Végpont	`GET /api/quizzes/{id}/start`
Leírás	Annak tesztelése, hogy a diák megkapja-e a vizsga kérdéseit, és hogy az adatok biztonságosan (a helyes megoldásokat elrejtve) érkeznek-e meg a frontend felé.
Előfeltétel	A vizsga státusza aktív (a jelenlegi időpont a start és end dátum közé esik), a kérést küldő diák pedig tagja a csoportnak, de még nem küldte be a vizsgát.
Teszt	

```
def test_submit_quiz(quizzes_client,
student_auth_headers, test_group, test_project):
    """Test a student submitting a quiz successfully."""
    now = timezone.now()
    quiz = Quiz.objects.create(
        group=test_group,
        project=test_project,
        date_start=now - timedelta(minutes=5),
        date_end=now + timedelta(hours=1)
    )

    block = test_project.blocks.first()
    correct_answer =
block.answers.filter(is_correct=True).first()

    payload = {
        "quiz_id": quiz.id,
        "answers": [
            {
                "block_id": block.id,
                "option_id": correct_answer.id,
                "answer_text": None
            }
        ]
    }

    response = quizzes_client.post("/submit/",
json=payload, headers=student_auth_headers)
```

	``` assert response.status_code == 200 data = response.json() assert data["percentage"] == "100.00" submission = Submission.objects.get(id=data["id"]) assert submission.score == 1.0 assert float(submission.percentage) == 100.0 ```
Várt Eredmény	HTTP 200-as státuszkód. A válasz JSON-ben szerepelnek a kérdések és a válaszlehetőségek, de a szenzitív adatok (pl. `is_correct` mező) a sémának köszönhetően nem kerülnek kiküldésre.
Eredmény	Sikeres